

TREES

MASTER NOTES

FAANG Placement	LeetCode	Codeforces	OA Rounds
10 Core Patterns	Complete Dry Runs	C++ Code Only	Brute → Optimal

PATTERNS COVERED

P01 Tree Traversals (DFS + BFS)	P06 LCA — Lowest Common Ancestor
P02 Binary Search Tree (BST)	P07 Tree DP — Diameter & Path Sum
P03 Tree Construction	P08 BST Validation & Properties
P04 Level Order / BFS Patterns	P09 Recursion on Trees
P05 Height, Balance & Symmetry	P10 Trie + Serialize/Deserialize

Structured for Pattern Recognition • Interview Communication • Contest Thinking

■ QUICK CHEAT SHEET — When To Use Which Pattern

Pattern	Trigger Keywords	Key Idea	TC / SC
DFS Traversals	Inorder/Preorder/Post, Path, Leaf	Recursive/Iterative stack	$O(N)/O(H)$
BFS / Level Order	Level-wise, Zigzag, Right view	Queue, process layer by layer	$O(N)/O(W)$
BST Operations	Search, Insert, Delete, Validate	Use BST property $L < \text{root} < R$	$O(H)/O(1)$
Tree Construction	Build from traversals, Clone	Preorder = root first	$O(N)/O(N)$
Height & Balance	Height, Diameter, Is Balanced	Post-order returns height	$O(N)/O(H)$
LCA	Lowest common ancestor, Distance	Both found = LCA	$O(N)/O(H)$
Tree DP	Max path sum, Rob III, Cameras	Return state from children	$O(N)/O(H)$
BST Validate	Valid BST, Kth smallest, Range	Pass min/max bounds	$O(N)/O(H)$
Recursion	Mirror, Flatten, Sum paths	Trust subtree result	$O(N)/O(H)$
Trie/Serialize	Word prefix, Serialize tree	char-node map / BFS queue	$O(N \cdot L)/O(N)$

■ Golden Decision Rules — 30-Second Pattern Recognition

Trigger / Clue	Pattern → Action
See inorder / preorder / postorder	DFS (P01) — recursive 3 lines or iterative with stack
See level-by-level / right view / zigzag	BFS with queue (P04) — process one level at a time
See BST + search/insert/validate	BST property: $\text{left} < \text{root} < \text{right}$ (P02/P08)
See height / balanced / diameter	Post-order DFS — return height up to parent (P05)
See LCA / distance between nodes	LCA recursion: if both found, current = LCA (P06)
See max path sum / rob tree / cameras	Tree DP — return best choice per node (P07)
See build tree from traversals	Preorder[0]=root, find in inorder, split (P03)
See serialize/deserialize tree	BFS (level order) or preorder with NULL markers (P10)
Interviewer says ' O(1) space '	Morris Traversal — use thread pointers instead of stack
Constraints $N \leq 10^5$, H balanced	$O(N)$ or $O(N \log N)$. Skewed tree → $O(N)$ height worst case

■ FAANG Company Frequency

Company	Most Asked Patterns	Frequency
Google	LCA, Serialize/Deserialize, Tree DP, BST	■■■■■
Meta	Diameter, Path Sum, BFS, LCA, Flatten	■■■■■
Amazon	Level Order, BST Validate, Inorder, Height	■■■■■
Microsoft	BST Operations, Traversals, Balanced Check	■■■■■
Uber	Trie, LCA, Tree DP, BFS patterns	■■■■■

Company	Most Asked Patterns	Frequency
Adobe	Traversals, Mirror, Construction from arrays	
Atlassian	Level Order, Path Sum, BST Range Query	

Pattern 01: Tree Traversals — DFS (Inorder / Preorder / Postorder)

Metric	Rating
FAANG Frequency	■■■■■ Very High – asked in EVERY interview
LeetCode Importance	■■■■■ Foundation of all tree problems
Codeforces Usefulness	■■■■■ High
Companies	Google, Meta, Amazon, Microsoft, Apple — every company
Difficulty	Easy → Medium
Problems	LC 94, 144, 145, 102, 105, 106, 114, 230, 98

■ Intuition

Three recursive orders define when you visit the ROOT relative to children. Inorder ($L \rightarrow \text{Root} \rightarrow R$) on a BST gives SORTED output. Preorder ($\text{Root} \rightarrow L \rightarrow R$) gives root first — used to reconstruct trees. Postorder ($L \rightarrow R \rightarrow \text{Root}$) processes children before parent — used for deletion, height. All three have $O(N)$ time and $O(H)$ space (recursion stack, H = height).

■ Brain Training Questions

- Why does inorder traversal of a BST give sorted order? Prove it.
- What is the iterative version of inorder? What data structure does it use?
- Preorder can reconstruct a tree with inorder — why do you need BOTH?
- Postorder: why are children always processed before parent?
- Morris Traversal: how do you do $O(1)$ space inorder using thread pointers?

■ Dry Run — Inorder on Example Tree

```
4
/ \
2 6
/ \ / \
1 3 5 7

Inorder ( $L \rightarrow \text{Root} \rightarrow R$ ): 1, 2, 3, 4, 5, 6, 7 ← sorted! ✓
Preorder ( $\text{Root} \rightarrow L \rightarrow R$ ): 4, 2, 1, 3, 6, 5, 7
Postorder ( $L \rightarrow R \rightarrow \text{Root}$ ): 1, 3, 2, 5, 7, 6, 4

Level Order (BFS): [4], [2,6], [1,3,5,7]
```

■ C++ Code — All Three Recursive Traversals

```
// ■ INORDER: L → Root → R (BST → sorted output)
void inorder(TreeNode* root, vector<int>& res) {
    if (!root) return;
    inorder(root->left, res);
    res.push_back(root->val); // visit root BETWEEN children
    inorder(root->right, res);
}

// ■ PREORDER: Root → L → R (clone tree, serialize)
void preorder(TreeNode* root, vector<int>& res) {
    if (!root) return;
    res.push_back(root->val); // visit root FIRST
    preorder(root->left, res);
    preorder(root->right, res);
}
```

```

}
// ■■■ POSTORDER: L → R → Root (delete tree, height)
void postorder(TreeNode* root, vector<int>& res) {
    if (!root) return;
    postorder(root->left, res);
    postorder(root->right, res);
    res.push_back(root->val); // visit root LAST
}
// All: Time O(N) Space O(H) – H=height, worst O(N) skewed

```

■ C++ Code — Iterative Inorder (LC 94)

```

// Iterative inorder – O(H) stack, no recursion
vector<int> inorderTraversal(TreeNode* root) {
    vector<int> res;
    stack<TreeNode*> st;
    TreeNode* curr = root;
    while (curr || !st.empty()) {
        while (curr) { st.push(curr); curr = curr->left; } // go left
        curr = st.top(); st.pop();
        res.push_back(curr->val); // visit
        curr = curr->right; // go right
    }
    return res;
}
// Time: O(N) Space: O(H)

```

■ Common Mistakes / Traps

- Iterative inorder: go left FIRST pushing to stack, then pop and go right — common mix-up
- Preorder iterative: push RIGHT child first, then LEFT (stack is LIFO)
- Postorder iterative: use two-stack trick OR reverse of (Root→R→L) preorder
- Always handle NULL root — most recursive solutions need if(!root) return base case
- Morris Traversal: restore tree after threading — forgetting this corrupts the tree

■ Interview Tips

- ✓ Always say which traversal and WHY — "I use postorder because I need children results first"
- ✓ Inorder + BST = sorted: if interviewer says "kth smallest" → inorder traversal (LC 230)
- ✓ Iterative > recursive in interviews when O(1) space asked → mention Morris Traversal
- ✓ "Can you do it without recursion?" → always yes, using explicit stack

■ Similar Problems

#	Problem	Difficulty	Traversal
1	94. Binary Tree Inorder	Easy	Inorder DFS
2	144. Binary Tree Preorder	Easy	Preorder DFS
3	145. Binary Tree Postorder	Easy	Postorder DFS
4	230. Kth Smallest in BST	Medium	Inorder BST
5	98. Validate BST	Medium	Inorder / Bounds
6	114. Flatten Tree to LL	Medium	Preorder in-place
7	105. Build from Pre+Inorder	Medium	Preorder+Inorder

Pattern 02: Binary Search Tree — Insert, Search, Delete

Metric	Rating
FAANG Frequency	Very High
LeetCode Importance	Must Know
Codeforces Usefulness	High
Companies	Amazon, Microsoft, Google, Meta
Difficulty	Easy → Medium
Problems	LC 700, 701, 450, 230, 235, 236, 270, 272, 285

■ Intuition

BST property: for every node, ALL left descendants $<$ node $<$ ALL right descendants. This means: search/insert/delete are $O(H)$ — $O(\log N)$ for balanced, $O(N)$ worst-case skewed. Inorder traversal of BST always gives sorted output — this is the #1 BST fact in interviews. BST delete has 3 cases: leaf (just remove), one child (replace), two children (replace with inorder successor).

■ Dry Run — BST Delete Node (3 Cases)

```
Tree: 5 → left:3(left:2,right:4) → right:7(left:6,right:8)
```

Case 1: Delete LEAF (node 2): Just set parent->left = NULL

Case 2: Delete ONE-CHILD node (node 3, only has right child 4):
Replace node 3 with its child 4.
Result: 5(left:4, right:7)

Case 3: Delete TWO-CHILD node (node 5 = root):
Find inorder SUCCESSOR = leftmost of right subtree = 6
Copy 6 into root. Delete 6 from right subtree.
Result: 6(left:3, right:7(left:NULL,right:8)) ✓

■ C++ Code — Search in BST (LC 700)

```
// LC 700 — O(H) time, O(H) space recursion | O(1) iterative
TreeNode* searchBST(TreeNode* root, int val) {
    if (!root || root->val == val) return root;
    return val < root->val ? searchBST(root->left, val)
        : searchBST(root->right, val);
}

// Iterative version — O(1) space:
TreeNode* searchBST(TreeNode* root, int val) {
    while (root && root->val != val)
        root = (val < root->val) ? root->left : root->right;
    return root;
} // Time: O(H) Space: O(1)
```

■ C++ Code — Delete Node in BST (LC 450)

```
// LC 450 — Delete from BST (3 cases)
TreeNode* deleteNode(TreeNode* root, int key) {
    if (!root) return nullptr;
    if (key < root->val) root->left = deleteNode(root->left, key);
    else if (key > root->val) root->right = deleteNode(root->right, key);
    else { // FOUND — 3 cases
```

```

if (!root->left) return root->right; // case 1 & 2
if (!root->right) return root->left; // case 2
// Case 3: two children – find inorder successor (min of right)
TreeNode* succ = root->right;
while (succ->left) succ = succ->left;
root->val = succ->val; // copy successor value
root->right = deleteNode(root->right, succ->val); // delete succ
}
return root;
}
// Time: O(H) Space: O(H) recursion – H = height

```

■ Common Mistakes / Traps

- BST delete Case 3: copy successor VALUE into node, then delete successor — never swap nodes
- Inorder successor = leftmost of RIGHT subtree (not parent)
- Inorder predecessor = rightmost of LEFT subtree
- "Closest value in BST" (LC 270): binary search style — go left/right based on comparison
- BST from sorted array must pick MIDDLE as root — else you get a skewed tree

■ Interview Tips

- ✓ Always ask: "Is the BST guaranteed to be balanced?" — affects time complexity
- ✓ "Inorder of BST = sorted" — state this explicitly before solving any BST problem
- ✓ For "Kth smallest" → do inorder, count to K. For "Kth largest" → reverse inorder
- ✓ Convert sorted array to balanced BST → binary divide and conquer (LC 108)

■ Similar Problems

#	Problem	Difficulty	Pattern
1	700. Search in a BST	Easy	BST Search
2	701. Insert into BST	Medium	BST Insert
3	450. Delete Node in BST	Medium	BST Delete (3 cases)
4	230. Kth Smallest in BST	Medium	Inorder Count
5	108. Sorted Array to BST	Easy	Mid as Root
6	235. LCA of BST	Medium	BST LCA
7	270. Closest Value in BST	Easy	Binary Search

Pattern 03: Tree Construction — Build from Traversals

Metric	Rating
FAANG Frequency	■■■■■ High – Google/Meta favourite
LeetCode Importance	■■■■■ Essential
Codeforces Usefulness	■■■■ Moderate
Companies	Google, Meta, Amazon
Difficulty	Medium → Hard
Problems	LC 105, 106, 889, 108, 1008, 654

■ Intuition

Preorder: first element is ALWAYS the root. Inorder: everything left of root is LEFT subtree, right is RIGHT subtree. Algorithm: take preorder[0] as root → find it in inorder → split both arrays → recurse. Use a HashMap of (inorder value → index) for O(1) lookup instead of O(N) linear search.

■ Dry Run — Build Tree from Preorder + Inorder (LC 105)

```
preorder = [3, 9, 20, 15, 7]
inorder = [9, 3, 15, 20, 7]
```

Step 1: root = preorder[0] = 3
3 is at index 1 in inorder → left has 1 node [9], right has 3 nodes [15,20,7]

Step 2: LEFT subtree:
preorder[1..1]=[9], inorder[0..0]=[9] → root=9 (leaf)

Step 3: RIGHT subtree:
preorder[2..4]=[20,15,7], inorder[2..4]=[15,20,7]
root=20. 15 is left of 20, 7 is right of 20.

Result: 3

```

/  \
9    20
/  \
15  7 ✓

```

■ C++ Code — Build Tree Preorder + Inorder (LC 105)

```
// LC 105 – O(N) with HashMap, O(N²) without
unordered map<int,int> idx; // inorder value → index

TreeNode* build(vector<int>& pre, int preL, int preR,
int inL, int inR) {
    if (preL > preR) return nullptr;
    int rootVal = pre[preL];
    int mid = idx[rootVal]; // find root in inorder
    int leftSize = mid - inL; // size of left subtree
    TreeNode* root = new TreeNode(rootVal);
    root->left = build(pre, preL+1, preL+leftSize,
inL, mid-1);
    root->right = build(pre, preL+leftSize+1, preR,
mid+1, inR);
    return root;
}
```



```
TreeNode* buildTree(vector<int>& pre, vector<int>& in) {  
    for (int i=0;i<in.size();i++) idx[in[i]]=i;  
    return build(pre, 0, pre.size()-1, 0, in.size()-1);  
}  
// Time: O(N) Space: O(N)
```

■ Common Mistakes / Traps

- Without HashMap: $O(N^2)$ due to linear search — always build map first
- $\text{leftSize} = \text{mid} - \text{inL}$ (NOT mid itself) — common index error
- Postorder + Inorder (LC 106): $\text{root} = \text{postorder}[\text{LAST}]$, logic is mirrored
- Preorder + Postorder (LC 889): doesn't uniquely define a tree — but LC 889 guarantees unique
- Max Binary Tree (LC 654): find max in range as root $\rightarrow$ split $\rightarrow$ recurse (no preorder given)

■ Similar Problems

#	Problem	Difficulty	Key Idea
1	105. Build Tree Pre+Inorder	Medium	$\text{Pre}[0]=\text{root}$, split Inorder
2	106. Build Tree Post+Inorder	Medium	$\text{Post}[-1]=\text{root}$, mirrored
3	889. Build Tree Pre+Postorder	Medium	$\text{Pre}[0]=\text{root}$, $\text{Post}[-2]=\text{left root}$
4	108. Sorted Array to BST	Easy	Mid as root, recurse
5	1008. BST from Preorder	Medium	BST bounds trick
6	654. Maximum Binary Tree	Medium	Max as root, recurse

Pattern 04: Level Order BFS — Layer-by-Layer Traversal

Metric	Rating
FAANG Frequency	■■■■■ Highest — most visual tree pattern
LeetCode Importance	■■■■■ Essential
Codeforces Usefulness	■■■■■ High
Companies	Amazon, Google, Meta, Microsoft — appears in ~40% of tree rounds
Difficulty	Easy → Medium
Problems	LC 102, 107, 103, 199, 637, 515, 116, 117, 1161

■ Intuition

BFS uses a queue. Process all nodes at depth D before any at depth D+1. Key trick: `snapshot queue.size()` at the START of each level — this tells you how many nodes belong to the current level. Process exactly that many nodes, adding their children to the queue. Zigzag = alternate left-to-right and right-to-left per level.

■ Dry Run — Level Order (LC 102)

```
Tree: 3
/  \
9  20
/  \
15 7

queue=[3]. Level 1: size=1. Process 3 → add 9,20. res=[[3]]
queue=[9,20]. Level 2: size=2. Process 9(add nothing),20(add 15,7). res=[[3],[9,20]]
queue=[15,7]. Level 3: size=2. Process 15,7. res=[[3],[9,20],[15,7]] ✓
```

■ C++ Code — Level Order (LC 102)

```
// LC 102 — Level Order Traversal
vector<vector<int>> levelOrder(TreeNode* root) {
    if (!root) return {};
    vector<vector<int>> res;
    queue<TreeNode*> q;
    q.push(root);
    while (!q.empty()) {
        int sz = q.size(); // SNAPSHOT level size
        vector<int> level;
        for (int i = 0; i < sz; i++) {
            TreeNode* node = q.front(); q.pop();
            level.push_back(node->val);
            if (node->left) q.push(node->left);
            if (node->right) q.push(node->right);
        }
        res.push_back(level);
    }
    return res;
}
// Time: O(N) Space: O(W) — W = max width of tree
```

■ C++ Code — Zigzag Level Order (LC 103)

```
// LC 103 — Zigzag: alternate direction per level
vector<vector<int>> zigzagLevelOrder(TreeNode* root) {
    if (!root) return {};
```

```

vector<vector<int>> res;
queue<TreeNode*> q;
q.push(root);
bool leftToRight = true;
while (!q.empty()) {
    int sz = q.size();
    vector<int> level(sz);
    for (int i = 0; i < sz; i++) {
        TreeNode* node = q.front(); q.pop();
        int pos = leftToRight ? i : sz-1-i; // fill by direction
        level[pos] = node->val;
        if (node->left) q.push(node->left);
        if (node->right) q.push(node->right);
    }
    res.push_back(level);
    leftToRight = !leftToRight;
}
return res;
}
// Time: O(N) Space: O(W)

```

■ Common Mistakes / Traps

- NEVER call `q.size()` inside the for loop — snapshot it BEFORE the loop starts
- Right Side View (LC 199): last element of each level — not always the rightmost node
- Zigzag: use index-based fill (not `deque.push_front`) — simpler and less error-prone
- Connect Next Right Pointers (LC 116): use level order OR $O(1)$ space with prev pointer trick
- BFS null check: always check if `node->left` and `node->right` exist before pushing

■ Similar Problems

#	Problem	Difficulty	BFS Variant
1	102. Binary Tree Level Order	Medium	Classic BFS
2	103. Zigzag Level Order	Medium	Direction flag
3	199. Right Side View	Medium	Last of each level
4	637. Average of Levels	Easy	Sum per level
5	515. Max in Each Level	Medium	Max per level
6	116. Populate Next Right Ptrs	Medium	Level linking
7	1161. Max Level Sum	Medium	Sum, find max level

Pattern 05: Height, Balance, Diameter & Symmetry

Metric	Rating
FAANG Frequency	Very High
LeetCode Importance	Must Know
Codeforces Usefulness	High
Companies	Google, Meta, Amazon, Microsoft
Difficulty	Easy → Medium
Problems	LC 104, 110, 543, 101, 112, 226, 572

Intuition

These problems share a pattern: use POSTORDER (process children first, then compute parent's answer). Height = 1 + max(left height, right height). Diameter = max path through any node = left height + right height (no +1 for node itself). Balance check = return -1 if unbalanced, else height — single O(N) pass. Key: return useful info UP to the parent, not just boolean.

Dry Run — Diameter (LC 543)

```
Tree: 1
/  \
2    3
/  \
4    5

diameter = 0 (global var)

height(4) = 1, height(5) = 1
At node 2: left_h=1, right_h=1. diameter = max(0, 1+1) = 2. return 2.
height(3) = 1
At node 1: left h=2, right h=1. diameter = max(2, 2+1) = 3. return 3.

Answer = 3 (path: 4→2→5 or 4→2→1→3) ✓
```

C++ Code — Diameter of Binary Tree (LC 543)

```
// LC 543 - Diameter = longest path (in edges) between any two nodes
int ans = 0;
int dfs(TreeNode* root) { // returns HEIGHT
    if (!root) return 0;
    int L = dfs(root->left);
    int R = dfs(root->right);
    ans = max(ans, L + R); // diameter through this node
    return 1 + max(L, R); // height of this subtree
}
int diameterOfBinaryTree(TreeNode* root) { dfs(root); return ans; }
// Time: O(N) Space: O(H)
```

C++ Code — Check Balanced Tree (LC 110)

```
// LC 110 - Is Balanced? (-1 = unbalanced, else = height)
int check(TreeNode* root) {
    if (!root) return 0;
    int L = check(root->left); if (L == -1) return -1;
    int R = check(root->right); if (R == -1) return -1;
```

```

if (abs(L - R) > 1) return -1; // unbalanced here
return 1 + max(L, R); // return height
}
bool isBalanced(TreeNode* root) { return check(root) != -1; }
// Time: O(N) Space: O(H) — One pass! Naive O(N²) checks height separately

```

■ C++ Code — Symmetric Tree (LC 101)

```

// LC 101 — Mirror check: left subtree == mirror of right subtree
bool isMirror(TreeNode* l, TreeNode* r) {
    if (!l && !r) return true;
    if (!l || !r) return false;
    return (l->val == r->val)
        && isMirror(l->left, r->right) // outer pair
        && isMirror(l->right, r->left); // inner pair
}
bool isSymmetric(TreeNode* root) {
    return isMirror(root->left, root->right);
}
// Time: O(N) Space: O(H)

```

■ Common Mistakes / Traps

- Diameter: don't confuse NODES vs EDGES — diameter in edges = left_h + right_h (no +1)
- Balanced check: naive approach calls height() at every node → O(N²). Use -1 sentinel trick
- Height of empty tree = 0. Height of single node = 1.
- Symmetric check: outer pair = (left.left, right.right), inner = (left.right, right.left)
- Path Sum (LC 112): subtract val as you go down, check leaf when remaining == 0

■ Similar Problems

#	Problem	Difficulty	Pattern
1	104. Maximum Depth of Tree	Easy	Post-order height
2	543. Diameter of Binary Tree	Easy	Height → Diameter
3	110. Balanced Binary Tree	Easy	Height with -1 trick
4	101. Symmetric Tree	Easy	Mirror DFS
5	112. Path Sum	Easy	DFS subtract
6	226. Invert Binary Tree	Easy	Swap children
7	572. Subtree of Another Tree	Easy	Height + match

Pattern 06: LCA — Lowest Common Ancestor

Metric	Rating
FAANG Frequency	■■■■■ Google Favourite
LeetCode Importance	■■■■■ Must Know
Codeforces Usefulness	■■■■■ High
Companies	Google, Meta, Amazon, Microsoft, Uber
Difficulty	Medium
Problems	LC 236, 235, 1650, 1644, 1676, 1123

■ Intuition

LCA of p and q = the deepest node that has BOTH p and q as descendants. Algorithm: recurse down. If current node IS p or q , return it. After left and right subtrees return: if BOTH are non-null $\rightarrow$ current node is LCA. If only one is non-null $\rightarrow$ propagate that up. For BST: simpler — if both $p, q < \text{root}$ go left; if both $> \text{root}$ go right; else root is LCA.

■ Dry Run — LCA (LC 236)

```
Tree: 3
/  \
5    1
/ \  / \
6 2 0 8
/  \
7  4
```

Find LCA(5, 4):

At node 3: search both subtrees

Left subtree (root=5): $5 == p \rightarrow$ return 5 (stop searching this branch)

Right subtree (root=1): neither 5 nor 4 found $\rightarrow$ return NULL

At node 3: left=5, right=NULL $\rightarrow$ return 5 (propagate up)

LCA = 5 ✓ (5 is ancestor of itself and of 4)

■ C++ Code — LCA Binary Tree (LC 236)

```
// LC 236 — LCA of any Binary Tree
TreeNode* lowestCommonAncestor(TreeNode* root, TreeNode* p, TreeNode* q) {
    if (!root || root == p || root == q) return root;
    // Search both subtrees
    TreeNode* left = lowestCommonAncestor(root->left, p, q);
    TreeNode* right = lowestCommonAncestor(root->right, p, q);
    // Both found in different subtrees  $\rightarrow$  current root is LCA
    if (left && right) return root;
    // One side has answer, propagate up
    return left ? left : right;
}
// Time: O(N) Space: O(H)
```

■ C++ Code — LCA of BST (LC 235)

```
// LC 235 — LCA of BST: exploit BST property for O(H) solution
TreeNode* lowestCommonAncestor(TreeNode* root, TreeNode* p, TreeNode* q) {
```

```
while (root) {
    if (p->val < root->val && q->val < root->val)
        root = root->left; // both in left subtree
    else if (p->val > root->val && q->val > root->val)
        root = root->right; // both in right subtree
    else return root; // split point = LCA
}
return nullptr;
// Time: O(H) Space: O(1) iterative — much better than general LCA!
```

■ Common Mistakes / Traps

- LCA with parent pointers (LC 1650): use two sets or pointer-equality technique
- "p or q may not exist in tree" (LC 1644): must verify both found after recursion
- LCA of K nodes (LC 1676): same logic — if current node is in the set, return it
- Distance between two nodes = $\text{depth}(p) + \text{depth}(q) - 2 * \text{depth}(\text{LCA})$
- Don't confuse LCA with "deepest common node" — same thing, just different phrasing

■ Similar Problems

#	Problem	Difficulty	Variant
1	236. LCA of Binary Tree	Medium	Classic LCA
2	235. LCA of BST	Medium	BST O(H) iterative
3	1650. LCA with Parent Pointer	Medium	Set or 2-pointer
4	1644. LCA if nodes may not exist	Medium	Both must be found
5	1676. LCA of K nodes	Medium	Set of targets
6	1123. Deepest Leaves LCA	Medium	Depth-based LCA

Pattern 07: Tree DP — Max Path Sum, House Robber, Camera

Metric	Rating
FAANG Frequency	Very High — Hard FAANG problems
LeetCode Importance	Must Know for Senior Roles
Codeforces Usefulness	Moderate
Companies	Google, Meta, Amazon — Hard round problems
Difficulty	Medium → Hard
Problems	LC 124, 337, 968, 1372, 979, 834

■ Intuition

Tree DP = DP where state is passed UP from children to parent (postorder). At each node, you receive results from left and right children, make a LOCAL decision, and return what the parent needs. Key: the function signature defines the DP state. Often return a pair/tuple (e.g., rob/don't rob) or use global variable for the answer.

■ C++ Code — Maximum Path Sum (LC 124)

```
// LC 124 — Max path sum (path can go through any node)
// Key: path through a node = left gain + node->val + right gain
int maxSum = INT_MIN;

int gain(TreeNode* node) {
    if (!node) return 0;
    int L = max(0, gain(node->left)); // only extend if positive
    int R = max(0, gain(node->right));
    maxSum = max(maxSum, L + node->val + R); // update global answer
    return node->val + max(L, R); // return best single-arm to parent
}

int maxPathSum(TreeNode* root) { gain(root); return maxSum; }
// Time: O(N) Space: O(H)
```

■ C++ Code — House Robber III (LC 337)

```
// LC 337 — Rob tree: no two adjacent nodes. DP state: {rob, skip}
pair<int,int> dp(TreeNode* node) { // returns {rob this, skip this}
    if (!node) return {0, 0};
    auto [lRob, lSkip] = dp(node->left);
    auto [rRob, rSkip] = dp(node->right);
    // If we rob this: must skip both children
    int rob = node->val + lSkip + rSkip;
    // If we skip this: take best of each child
    int skip = max(lRob, lSkip) + max(rRob, rSkip);
    return {rob, skip};
}

int rob(TreeNode* root) {
    auto [r, s] = dp(root);
    return max(r, s);
}
// Time: O(N) Space: O(H) — No memo map needed!
```

■ C++ Code — Binary Tree Cameras (LC 968)

```
// LC 968 — Min cameras to monitor all nodes
// State: 0=uncovered, 1=has camera, 2=covered (no camera)
int cameras = 0;
```



```

int dfs(TreeNode* node) {
    if (!node) return 2; // NULL = covered
    int L = dfs(node->left), R = dfs(node->right);
    if (L == 0 || R == 0) { cameras++; return 1; } // child uncovered
    if (L == 1 || R == 1) return 2; // child has camera, covers us
    return 0; // both children covered but no camera on them
}

int minCameraCover(TreeNode* root) {
    return (dfs(root) == 0 ? 1 : 0) + cameras;
} // Time: O(N) Space: O(H)

```

■ Common Mistakes / Traps

- Path Sum: path does NOT have to pass through root — use global var updated at each node
- Path extends only if child gain > 0 — use $\max(0, \text{gain}(\text{child}))$ to cut negative branches
- House Robber III: return PAIR not single value — single value loses information
- Cameras: leaf nodes should NOT have cameras (greedy: push cameras to parents)
- Return value = what parent NEEDS, answer = global variable updated during recursion

■ Similar Problems

#	Problem	Difficulty	DP State
1	124. Binary Tree Max Path Sum	Hard	Global max, return arm
2	337. House Robber III	Medium	Pair {rob, skip}
3	968. Binary Tree Cameras	Hard	3-state per node
4	1372. Longest Zigzag Path	Medium	Return {left, right} len
5	979. Distribute Coins	Medium	Excess coins per subtree
6	834. Sum of Distances in Tree	Hard	Rerooting DP

Pattern 08: BST Validation, Range Queries & Inorder Properties

Metric	Rating
FAANG Frequency	■■■■■ High
LeetCode Importance	■■■■■ Essential
Codeforces Usefulness	■■■■ Moderate
Companies	Amazon, Microsoft, Google
Difficulty	Easy → Medium
Problems	LC 98, 99, 530, 501, 173, 1038, 285

■ Intuition

Validate BST: pass min and max bounds down the tree. Every node must be strictly within (min, max). Going left: update max = current val. Going right: update min = current val. The inorder property links BST to sorted arrays — many BST problems reduce to: "do inorder, solve the sorted array version".

■ C++ Code — Validate BST (LC 98)

```
// LC 98 – Validate BST with min/max bounds
bool valid(TreeNode* node, long lo, long hi) {
    if (!node) return true;
    if (node->val <= lo || node->val >= hi) return false;
    return valid(node->left, lo, node->val) // left: upper bound = val
    && valid(node->right, node->val, hi); // right: lower bound = val
}

bool isValidBST(TreeNode* root) {
    return valid(root, LONG_MIN, LONG_MAX);
}

// Time: O(N) Space: O(H)
// Note: use LONG_MIN/MAX (not INT) to handle INT_MIN/MAX node values!
```

■ C++ Code — Recover BST — 2 Swapped Nodes (LC 99)

```
// LC 99 – Exactly 2 nodes swapped in BST. Find and restore.
// In inorder, swapped nodes appear as TWO inversions: a > b
TreeNode *first=nullptr, *second=nullptr, *prev=nullptr;

void inorder(TreeNode* node) {
    if (!node) return;
    inorder(node->left);
    if (prev && prev->val > node->val) {
        if (!first) first = prev; // first inversion: take the larger
        second = node; // always update second
    }
    prev = node;
    inorder(node->right);
}

void recoverTree(TreeNode* root) {
    inorder(root);
    swap(first->val, second->val);
}

// Time: O(N) Space: O(H) – O(1) space with Morris Traversal
```

■ Common Mistakes / Traps

- Validate BST: use LONG_MIN / LONG_MAX as initial bounds — INT_MIN/MAX fails for edge values

- Validate BST: inorder approach fails if you only compare adjacent — use bounds approach
- Recover BST: adjacent swap = 1 inversion. Non-adjacent = 2 inversions. second always updated.
- BST Iterator (LC 173): use stack-based inorder generator — not array ($O(N)$ space)
- Greater Sum Tree (LC 1038): REVERSE inorder ($R \rightarrow \text{Root} \rightarrow L$) with running sum

■ Similar Problems

#	Problem	Difficulty	Key Idea
1	98. Validate BST	Medium	Min/Max Bounds
2	99. Recover BST	Hard	Inorder Inversions
3	530. Min Abs Diff in BST	Easy	Inorder Adjacent
4	173. BST Iterator	Medium	Stack Inorder
5	1038. BST to Greater Sum Tree	Medium	Reverse Inorder
6	285. Inorder Successor in BST	Medium	Binary Search + Track

Pattern 09: Recursion on Trees — Path, Sum & Transformation

Metric	Rating
FAANG Frequency	■■■■■ High
LeetCode Importance	■■■■■ Essential
Codeforces Usefulness	■■■■ Moderate
Companies	Google, Meta — tests deep recursion thinking
Difficulty	Easy → Medium
Problems	LC 113, 129, 437, 257, 572, 1448, 226

■ Intuition

Root-to-leaf paths: carry accumulated state (sum, path) downward, record at leaves. Path Sum III (LC 437): any-node paths → use prefix sum + HashMap passed via DFS. Recursive transformation: swap left/right children (invert), flatten to LL. Mental model: define what your function receives (accumulated) and what it returns (subtree result).

■ C++ Code — Path Sum III (LC 437) — Prefix Sum on Tree

```
// LC 437 – Count paths (any start/end) with sum = targetSum
// Key: prefix sum on tree path, same as subarray sum = k
int pathSum(TreeNode* root, int target) {
    unordered_map<long, int> mp = {{0, 1}};
    return dfs(root, 0, target, mp);
}

int dfs(TreeNode* node, long curr, int target,
        unordered_map<long, int>& mp) {
    if (!node) return 0;
    curr += node->val;
    int count = mp[curr - target]; // paths ending here with sum=target
    mp[curr]++;
    count += dfs(node->left, curr, target, mp);
    count += dfs(node->right, curr, target, mp);
    mp[curr]--; // BACKTRACK – remove from map on return
    return count;
}

// Time: O(N) Space: O(N) – same as subarray sum but on tree
```

■ C++ Code — All Root-to-Leaf Paths (LC 257)

```
// LC 257 – Collect all root-to-leaf paths
void dfs(TreeNode* node, string path, vector<string>& res) {
    if (!node) return;
    path += to_string(node->val);
    if (!node->left && !node->right) { res.push_back(path); return; }
    path += "->";
    dfs(node->left, path, res);
    dfs(node->right, path, res);
}

vector<string> binaryTreePaths(TreeNode* root) {
    vector<string> res;
    dfs(root, "", res);
    return res;
}

// Time: O(N*H) Space: O(H) for path string
```

■ Common Mistakes / Traps

- Path Sum III: MUST backtrack (mp[curr]--) after returning — else sibling paths get wrong counts
- Root-to-leaf sum: leaf = node with NO children (both null), not just "no left child"
- Sum of Root-to-Leaf Numbers (LC 129): carry running number $\times 10 + \text{val}$ down
- Count Good Nodes (LC 1448): carry max-so-far downward, node is good if $\text{val} \geq \text{max}$
- String paths: build string top-down. Prefer stringstream or int vector to avoid $O(N^2)$ concat

■ Similar Problems

#	Problem	Difficulty	Pattern
1	112. Path Sum	Easy	DFS subtract
2	113. Path Sum II	Medium	Backtrack + collect
3	437. Path Sum III	Medium	Prefix Sum on Tree
4	257. Binary Tree Paths	Easy	DFS with string
5	129. Sum Root to Leaf Numbers	Medium	DFS with number
6	1448. Count Good Nodes	Medium	DFS carry max
7	988. Smallest Str from Leaf	Medium	DFS + compare

Pattern 10: Trie + Serialize & Deserialize Trees

Metric	Rating
FAANG Frequency	■■■■■ High – Frequently Hard round question
LeetCode Importance	■■■■■ Must Know
Codeforces Usefulness	■■■■ Moderate
Companies	Google, Meta, Uber, Microsoft
Difficulty	Medium → Hard
Problems	LC 297, 449, 208, 212, 211, 745, 336

■ Intuition: Serialize / Deserialize

Serialize = convert tree to string/array for storage. Deserialize = reconstruct tree from that representation. Two main approaches: (1) BFS level-order with NULL markers — easy to understand. (2) Preorder DFS with NULL markers — more elegant, fewer characters. BST serialize is simpler — just preorder without NULLs (BST property reconstructs it).

■ C++ Code — Serialize & Deserialize Binary Tree (LC 297)

```
// LC 297 – Preorder DFS with "#" for null
string serialize(TreeNode* root) {
    if (!root) return "#,";
    return to_string(root->val)+","+serialize(root->left)+serialize(root->right);
}

TreeNode* deserialize(string data) {
    queue<string> q;
    stringstream ss(data);
    string tok;
    while (getline(ss, tok, ',')) q.push(tok);
    return build(q);
}

TreeNode* build(queue<string>& q) {
    string val = q.front(); q.pop();
    if (val == "#") return nullptr;
    TreeNode* node = new TreeNode(stoi(val));
    node->left = build(q);
    node->right = build(q);
    return node;
}

// Time: O(N) Space: O(N)
```

■ Intuition: Trie (Prefix Tree)

A Trie stores strings character by character. Each node = one character, edges = next character. Root is empty. Insert: traverse/create char nodes. Search: follow chars, check isEnd flag. Prefix search: same but no isEnd check. Time: $O(L)$ per operation where L = word length.

■ C++ Code — Implement Trie (LC 208)

```
// LC 208 – Trie (Prefix Tree)
struct TrieNode {
    TrieNode* ch[26] = {};
    bool isEnd = false;
};

class Trie {
    TrieNode* root = new TrieNode();
public:
```

```

void insert(string w) {
    TrieNode* cur = root;
    for (char c : w) {
        if (!cur->ch[c-'a']) cur->ch[c-'a'] = new TrieNode();
        cur = cur->ch[c-'a'];
    }
    cur->isEnd = true;
}

bool search(string w) {
    TrieNode* cur = root;
    for (char c : w) {
        if (!cur->ch[c-'a']) return false;
        cur = cur->ch[c-'a'];
    }
    return cur->isEnd;
}

bool startsWith(string p) {
    TrieNode* cur = root;
    for (char c : p) {
        if (!cur->ch[c-'a']) return false;
        cur = cur->ch[c-'a'];
    }
    return true; // no isEnd check for prefix
}
};

// Insert/Search/Prefix: O(L) time, O(L) space per word

```

Common Mistakes / Traps

- **Serialize:** don't forget NULL markers ("##") — without them you can't uniquely deserialize
- **Deserialize** uses a QUEUE (not vector index) — consume tokens in order during recursion
- **Trie search vs startsWith:** search requires isEnd=true, prefix does NOT
- **Word Search II (LC 212):** build Trie from word list, then DFS on board with backtrack
- **Trie memory:** 26 pointers per node = expensive. Use unordered_map for sparse alphabets

Interview Tips

- ✓ **Serialize:** preorder with # is shorter code than BFS — prefer it unless asked for level-order
- ✓ **Trie follow-up:** "Add wildcard search" → DFS on Trie with '.' matching any child
- ✓ **"Why Trie over HashMap?"** → Trie gives O(L) prefix operations, HashMap O(L) hash only
- ✓ **Word Break, Palindrome Pairs** — both reducible to Trie + clever traversal

Similar Problems

#	Problem	Difficulty	Type
1	297. Serialize/Deserialize BT	Hard	Preorder + queue
2	449. Serialize/Deserialize BST	Medium	Preorder no nulls
3	208. Implement Trie	Medium	Classic Trie
4	212. Word Search II	Hard	Trie + DFS on grid
5	211. Design Add-Search Words	Medium	Trie + wildcard DFS
6	336. Palindrome Pairs	Hard	Trie + reverse lookup

■ Learning Roadmap & Revision Guide

Week	Patterns	Key Problems	Goal
Week 1	P01 Traversals, P05 Height/Balance	LC 94,144,145,104,543,101	Tree DFS basics
Week 2	P04 BFS, P02 BST	LC 102,103,199,700,701,450	BFS + BST ops
Week 3	P06 LCA, P08 BST Validate	LC 236,235,98,99,230,530	LCA + BST props
Week 4	P03 Construction, P09 Path DP	LC 105,106,437,113,129	Build + path sums
Week 5	P07 Tree DP, P10 Trie/Serial	LC 124,337,968,297,208	Hard patterns
Week 6	All patterns	LeetCode Tree Tag full	Speed + accuracy

■ Must-Solve LeetCode Problems (Ordered by Importance)

#	Problem	Difficulty	Pattern
1	104. Max Depth of Binary Tree	Easy	P05 — Foundation
2	226. Invert Binary Tree	Easy	P09 Recursion
3	94. Binary Tree Inorder	Easy	P01 Traversal
4	102. Binary Tree Level Order	Medium	P04 BFS
5	543. Diameter of Binary Tree	Easy	P05 Height→Diameter
6	110. Balanced Binary Tree	Easy	P05 -1 trick
7	100. Same Tree	Easy	P09 Recursion
8	572. Subtree of Another Tree	Easy	P09+P05
9	235. LCA of BST	Medium	P06 BST LCA
10	236. LCA of Binary Tree	Medium	P06 General LCA
11	105. Build from Pre+Inorder	Medium	P03 Construction
12	98. Validate BST	Medium	P08 Bounds
13	199. Right Side View	Medium	P04 BFS Last
14	230. Kth Smallest in BST	Medium	P02 Inorder
15	112. Path Sum	Easy	P09 DFS
16	437. Path Sum III	Medium	P09 Prefix Sum
17	297. Serialize/Deserialize	Hard	P10 Preorder
18	124. Max Path Sum	Hard	P07 Tree DP
19	337. House Robber III	Medium	P07 Pair DP
20	208. Implement Trie	Medium	P10 Trie

■ Pattern Dependency Map — Learn In This Order

Pattern	Depends On	Unlocks
P01 DFS Traversals	None — start here	P03 Construction, P06 LCA, P09 Paths
P04 BFS Level Order	P01 (tree traversal concept)	P06 LCA (via levels), P10 Serialize
P02 BST Operations	P01 Inorder + BST property	P08 Validate, P06 BST LCA
P05 Height + Balance	P01 Postorder pattern	P07 Tree DP (same postorder)
P06 LCA	P01 DFS + P02 BST	P07 DP (uses LCA concepts)
P03 Construction	P01 Traversals (all three)	P10 Serialize (reverse)
P07 Tree DP	P05 (postorder pattern)	Advanced Hard problems
P08 BST Validate	P01 Inorder + P02 BST	P99 Recover BST, Iterator
P09 Recursion Paths	P01 DFS + P05 (carry state)	P07 Tree DP extension
P10 Trie + Serial	P01 Preorder + P04 BFS	Word Search, Palindrome Pairs

■ Universal Problem-Solving Framework — 7-Step Process

Step 1 — Keywords	Level/layer → BFS. Path/root-to-leaf → DFS carry. Max/diameter → Postorder. BST → O(H) exploit. Cycle/duplicate → none in trees.
Step 2 — Constraints	O(1) space → Morris Traversal. Balanced BST → O(log N). Skewed → O(N). Large N with paths → avoid O(N ²) string concat.
Step 3 — Draw	Always draw the tree. Label left/right, mark null nodes. Draw what the recursion returns at each level.
Step 4 — Brute First	Naïve height at every node → O(N ²). Then optimize: one-pass with -1 sentinel.
Step 5 — Optimize	Repeated subproblem? → Memo. Multiple values needed per node? → Return pair/tuple. Any-path problem? → Prefix sum on tree.
Step 6 — Edge Cases	Empty tree (root=NULL), single node, left/right skewed tree, duplicate values in BST, all negative values for path sum.
Step 7 — Communicate	"I'll use postorder because I need children's results before computing the parent's answer" — shows traversal awareness.

■ FAANG vs OA vs Codeforces Priority

Context	Most Important Patterns	Notes
FAANG Interview	P07 Tree DP, P06 LCA, P10 Serialize, P04 BFS	Explanation required
OA / Online Assessment	P01 Traversals, P05 Height, P04 BFS, P02 BST	Speed + accuracy
Codeforces Div2 A-C	P05 Height/Balance, P01 DFS, P04 BFS	Straightforward impl
Codeforces Div2 D-E	P07 Tree DP, Centroid Decomp, P06 LCA	Advanced tree theory

The goal is NOT to memorize solutions. It is to recognize patterns instantly and adapt them.

When you see a new tree problem, ask: Which traversal order do I need? What should each node return?